



Laporan Implementasi

Opsi B: Encrypted Token Storage

1. Edge Function Proxy

Per-Camera Auth Token untuk CCTV Streaming



IMPLEMENTED



BUILD PASS



8 LAYERS



BACKWARD COMPAT



Tanggal:

16 Februari 2026



Versi:

1.0.0



Files:

2 created + 6 modified



Stack:

React 18 · Vite · Supabase · Vercel · Cloudflare



Daftar Isi

- 1 Ringkasan Eksekutif
- 2 Arsitektur & Data Flow
- 3 File Baru (Created)
- 4 File yang Dimodifikasi
- 5 Model Keamanan (8 Layers)
- 6 Panduan Deployment
- 7 Backward Compatibility
- 8 Checklist Verifikasi

1 Ringkasan Eksekutif

✓ Implementasi Selesai — 10/10 Tasks

Build Vite berhasil: exit code 0 · 1592 modules · 12.36s.

Per-camera auth token end-to-end: UI form → encrypt DB → proxy stream via Edge Function.



Tujuan



Setiap kamera punya **auth token unik** — tidak sharing global key



Token **tidak pernah terekspos ke browser** — encrypted at rest, decrypted server-side



Operator cukup **isi form di dashboard** — tidak perlu edit .env



Pendekatan (3 Layer)



Layer 1: Database

AES-256 encrypt via
pgp_sym_encrypt.
Kolom di-REVOKE dari anon/au-
thenticated.



Layer 2: Edge Function

Proxy server-side: decrypt token,
validasi JWT, forward MJPEG stream.

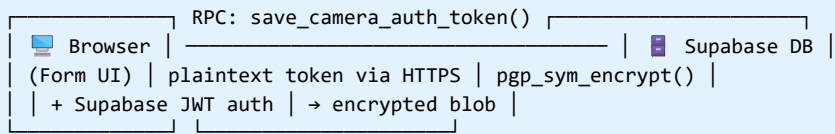


Layer 3: Frontend

Form write-only (password field).
CCTVFeed dual-mode: proxy /
legacy.

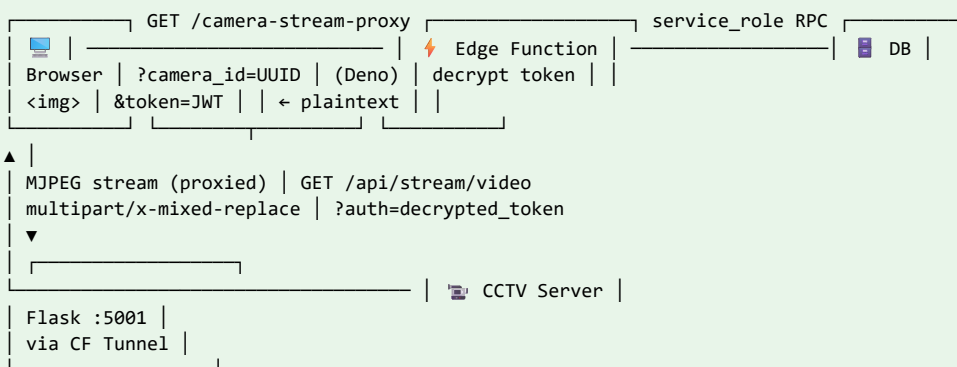
2 Arsitektur & Data Flow

Alur Penyimpanan Token



✓ Token plaintext hanya ada in-transit (HTTPS). Di DB = AES-256 encrypted.

Alur Streaming (Runtime)



🔒 Browser TIDAK PERNAH melihat auth token CCTV server. Hanya kirim JWT Supabase.

3 File Baru (Created)

3.1 Database Migration

 [supabase/migrations/008_add_camera_stream_auth.sql](#)

EXT CREATE EXTENSION IF NOT EXISTS pgcrypto
COL stream_url TEXT — URL endpoint CCTV server per kamera
COL auth_token_encrypted TEXT — token AES-256 encrypted
COL has_auth_token BOOLEAN GENERATED — safe boolean flag
RPC save_camera_auth_token() — encrypt & store, cek role
RPC get_camera_stream_credentials() — decrypt (service_role only)
RPC remove_camera_auth_token() — hapus token, cek role
SEC REVOKE SELECT (auth_token_encrypted) dari anon/authenticated

WAJIB: Set Encryption Key Dulu

Jalankan di SQL Editor SEBELUM pakai RPC:

```
ALTER DATABASE postgres SET app.encryption_key = 'your-key-min-32-chars';
```

3.2 Supabase Edge Function

 [supabase/functions/camera-stream-proxy/index.ts](#)

AUTH JWT validation: Authorization header atau ?token= query param
AUTHZ Tenant membership check via user_tenants table
CRYPTO Decrypt token via get_camera_stream_credentials() RPC
PROXY MJPEG stream pipe: upstream body → browser response
VALID UUID format check + stream type whitelist
CORS Preflight handling + error sanitization

4 File yang Dimodifikasi (6 files)

`src/hooks/useCameras.js`

-  `stream_url`, `has_auth_token` ditambahkan ke `SELECT` (BUKAN `auth_token_encrypted`)
-  `saveAuthToken(cameraId, authToken, streamUrl)` — panggil RPC encrypt & store
-  `removeAuthToken(cameraId)` — panggil RPC hapus token
-  `addCamera()` — auto-call `saveAuthToken()` jika token diberikan

`src/services/api.js`

-  `SUPABASE_URL` dari env var `VITE_SUPABASE_URL`
-  `getCameraProxyStreamUrl(cameraId, type, jwt)` — build proxy URL
-  Legacy functions (`getStreamUrl`, dll.) tetap ada — backward compatible

`src/pages/CamerasPage.jsx`

-  Section baru “Stream Configuration” di modal form
-  Field `stream_url` (type=url) + `auth_token` (type=password, autoComplete=off)
-  Badge “🔒 Tersimpan” saat edit kamera yang punya token
-  Kolom “Stream” baru di tabel data: status + lock icon

Write-Only Pattern

Saat edit, field `auth_token` **selalu kosong**. Placeholder: “Kosongkan jika tidak ingin mengubah”. Token lama **TIDAK PERNAH** dikembalikan ke browser. Hanya `has_auth_token` boolean sebagai indikator.

`src/components/CCTVFeed.jsx`

-  Props baru: `cameraId` (UUID) + `cameraData` (dari `useCameras`)
-  **Dual-mode**: proxy URL (jika `cameraId` + `stream_url`) ATAU legacy global URL
-  JWT auto-fetch via `supabase.auth.getSession()` + `onAuthStateChange`
-  Lock icon di header jika `has_auth_token` = true

`src/pages/LiveStreamingPage.jsx`

-  Camera selector dropdown: “Global Stream (Legacy)” + DB cameras
-  Auto-select kamera pertama yang streamable
-  Icon 🔒 di dropdown untuk kamera dengan auth token

`src/pages/DashboardOverview.jsx`

-  `CameraThumbnail` — dual mode: legacy cam number ATAU DB `cameraRecord`
-  `CCTVPreviewSection` — max 4 DB cameras jika ada, fallback ke grid [1,2,3,4]
-  Proxy snapshot URL via Edge Function untuk DB cameras

5 Model Keamanan — 8 Defense Layers

#		Mekanisme	Ancaman Dicegah
1		HTTPS everywhere (CF Tunnel + Supabase + Vercel)	Sniffing / MITM
2		Supabase JWT (short-lived access token, 1 jam)	Unauthorized access
3		Tenant membership check (Edge Fn + RLS)	Cross-tenant data leak
4		user_has_role() — owner/admin/operator only	Privilege escalation
5		AES-256 via pgp_sym_encrypt (pgcrypto)	DB breach → token aman
6		REVOKE SELECT (auth_token_encrypted)	Direct SELECT bypass
7		Write-only UI — hanya has_auth_token boolean	Client-side token leak
8		Edge Function proxy — decrypt server-side only	DevTools exposure

🔪 Threat Model

Threat	Mitigasi	Risk
JWT sniffed dari URL query param	JWT 1 jam lifetime; HTTPS only	LOW
DB breach → read encrypted token	AES-256 + separate encryption key	LOW
Compromised Edge Function env	SERVICE_ROLE_KEY di Supabase infra saja	MED
Cross-tenant via camera_id guessing	Tenant membership check post-decrypt	LOW
Token leak via form autocomplete	autoComplete="off" on password field	LOW
MJPEG hijack dari proxy response	CORS + JWT wajib per-request	LOW

6 Panduan Deployment — 5 Steps

1

Set Encryption Key (SQL Editor)

```
ALTER DATABASE postgres SET app.encrypted_key = 'your-strong-random-key-min-32-chars';
```

Tanpa ini, semua RPC save/get token akan **GAGAL**.

2

Jalankan Migration

```
supabase db push --project-ref <your-project-ref>
```

Atau copy-paste isi `008_add_camera_stream_auth.sql` ke SQL Editor.

3

Deploy Edge Function

```
supabase functions deploy camera-stream-proxy --project-ref <your-project-ref>
```

SUPABASE_URL dan SERVICE_ROLE_KEY otomatis tersedia di environment Edge Function.

4

Redeploy Dashboard

```
git add -A && git commit -m "feat(security): per-camera auth" && git push
```

Vercel akan auto-deploy. Atau manual: `cd dashboard && vercel --prod`

5

Tambah Kamera di Dashboard

Buka **Cameras** → **Add Camera** → isi **Stream URL** + **Auth Token** → Save.

Token terenkripsi di DB. Badge  muncul di tabel. Selesai!

7 Backward Compatibility

✓ Fully Backward Compatible

Jika tidak ada kamera DB dengan `stream_url`, sistem otomatis fallback ke global stream URL dari `VITE_API_URL` + `VITE_SERVER_AUTH_KEY`. Tidak ada breaking change.

Komponen	● Tanpa DB Camera	● Dengan DB Camera
CCTVFeed	Global stream URL (legacy)	Proxy URL via Edge Function
LiveStreamingPage	Tidak ada dropdown	Dropdown selector + proxy
DashboardOverview	Grid [1,2,3,4]	Grid dari DB cameras
CamerasPage	Form standar	Form + Stream Configuration
api.js	<code>getStreamUrl()</code> tetap ada	<code>getCameraProxyStreamUrl()</code> baru

8 Checklist Verifikasi

#	Task	Status
1	Migration 008: columns + pgcrypto + 3 RPCs + column REVOKE	DONE ✓
2	Edge Function: JWT + decrypt + tenant check + MJPEG proxy	DONE ✓
3	Hook: saveAuthToken() + removeAuthToken() + new columns	DONE ✓
4	UI: stream config form + write-only token + table column	DONE ✓
5	API: getCameraProxyStreamUrl() function	DONE ✓
6	CCTVFeed: dual-mode (proxy / legacy) + JWT fetch	DONE ✓
7	LiveStreamingPage: camera dropdown selector	DONE ✓
8	DashboardOverview: DB camera thumbnails	DONE ✓
9	Build: vite build exit code 0 (1592 modules)	DONE ✓
10	Documentation & deployment guide	DONE ✓

✓ Implementasi Opsi B — 100% Complete

BUILD ✓ 8 FILES 8 LAYERS BACKWARD ↔

Token encrypted at rest · Decrypted only by Edge Function · Browser never sees CCTV server key